Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №_

«Название лабораторной работы»

Выполнил:
студент группы 3825Б1ФИсп1
Курочкин М.Н.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение		3
1.	Постановка задачи	4
2.	Руководство пользователя	5
3.	Руководство программиста	6
3.1.	Описание структуры программы	6
3.2.	Описание структур данных	6
3.3.	Описание алгоритмов	6
4.	Результаты экспериментов и тестирования	7
Заключение		8
Литература		9
Приложение		10

Введение

Множества, элементами которых могут быть только значения из заранее заданного конечного набора, удобно хранить в виде битовых полей: каждому возможному элементу сопоставляется один бит, показывающий его наличие во множестве. Это экономит память и сводит операции над множествами к побитовым операциям.

В работе на C++ реализованы классы TBitField (хранение битового поля, доступ к битам, операции &, |, ~, сравнение, ввод/вывод) и TSet (множество на основе битового поля: добавление, удаление, проверка принадлежности, объединение, пересечение, дополнение).

Цель работы — изучить представление множеств через битовые поля, освоить битовые операции и написание тестов с помощью GoogleTest. Корректность проверяется набором тестов.

1. Постановка задачи

Требуется реализовать класс TBitField, представляющий собой набор битов с номерами от 0 до BitLen-1, и класс TSet, реализующий множество на основе битового поля. Необходимо:

1. Реализовать конструкторы, деструктор, конструктор копирования и оператор присваивания для обоих классов.

2. Реализовать методы доступа к битам (SetBit, ClrBit, GetBit, GetLength).

3. Перегрузить операции сравнения (==, !=), побитовые операции (&, |, ~) для TBitField.

4. Для TSet реализовать методы InsElem, DelElem, IsMember, GetMaxPower, операции объединения (+), пересечения (*), дополнения (~), а также операции с элементом (+, -).

5. Реализовать ввод/вывод в поток для обоих классов.

6. Написать тесты с использованием фреймворка Google Test, покрывающие все методы и граничные случаи.

2. Руководство пользователя

Программа представляет собой библиотеку классов, которая может быть использована в других приложениях. Пример использования:

```cpp
#include "rl_tbitfield.h"
#include "rl_tset.h"
#include <iostream>

int main() {
    // Работа с битовым полем
    TBitField bf(8);
    bf.SetBit(0);
    bf.SetBit(3);
    std::cout << "Битовое поле: " << bf << std::endl; // Вывод: 10010000

    // Работа с множеством
    TSet s1(10);
    s1.InsElem(2);
    s1.InsElem(5);
    std::cout << "Множество: " << s1 << std::endl; // Вывод: {2, 5}

    TSet s2(10);
    s2.InsElem(5);
    s2.InsElem(7);
    TSet unionSet = s1 + s2;
    std::cout << "Объединение: " << unionSet << std::endl; // Вывод: {2, 5, 7}

    TSet interSet = s1 * s2;
    std::cout << "Пересечение: " << interSet << std::endl; // Вывод: {5}

    TSet compSet = ~s1;
    std::cout << "Дополнение: " << compSet << std::endl; // Вывод: {0, 1, 3, 4, 6, 7, 8, 9}

    return 0;
}
```

Варианты использования:

- Создание битового поля заданной длины (положительное целое число).
- Установка/сброс/чтение конкретного бита по индексу (индекс должен быть в диапазоне [0, BitLen-1]).
- Выполнение побитовых операций над полями (результат имеет длину, равную максимальной из длин операндов).
- Создание множества с заданной максимальной мощностью.
- Добавление/удаление элементов, проверка принадлежности.
- Выполнение теоретико-множественных операций.
- Ввод/вывод битового поля в виде строки из '0' и '1', множества — в виде {элемент1, элемент2, ...}.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей:

1. tbitfield.h — объявление класса TBitField (интерфейс).
2. rl_tbitfield.h — реализация методов класса TBitField.
3. tset.h — объявление класса TSet.
4. rl_tset.h — реализация методов класса TSet.
5. test.cpp — модульные тесты для обоих классов с использованием Google Test.

Класс TSet агрегирует объект TBitField, используя его как хранилище характеристического вектора множества.

3.2. Описание структур данных

Класс TBitField:

1. int BitLen — длина битового поля (количество битов).
2. TELEM *pMem — указатель на динамический массив элементов типа unsigned int (32 бита).
3. int MemLen — количество элементов в массиве pMem.
4. typedef unsigned int TELEM — тип элемента памяти.

Класс TSet:

1. int MaxPower — максимальная мощность множества (универсальное множество).
2. TBitField BitField — битовое поле, хранящее характеристический вектор: бит i равен 1, если элемент i принадлежит множеству.

Глобальные функции:

operator>>, operator<< для TBitField и TSet — дружественные функции потокового ввода/вывода.

3.3. Описание алгоритмов

Класс TBitField:

1. Конструктор TBitField(int len):

   Проверяет len > 0, иначе выбрасывает std::invalid_argument.
   Вычисляет MemLen = (len + 31) / 32.
   Выделяет память new TELEM[MemLen]() (с инициализацией нулями).

2. GetMemIndex(n): возвращает n / 32 — индекс элемента массива для бита n.

3. GetMemMask(n): возвращает 1 << (n % 32) — маску для бита n.

4. SetBit(n): проверяет диапазон, выполняет pMem[GetMemIndex(n)] |= GetMemMask(n).

5. ClrBit(n): проверяет диапазон, выполняет pMem[GetMemIndex(n)] &= ~GetMemMask(n).

6. GetBit(n): проверяет диапазон, возвращает (pMem[GetMemIndex(n)] & GetMemMask(n)) != 0.

7. operator&: создаёт результат длиной max(BitLen1, BitLen2), выполняет побитовое И для общих элементов массива.

8. operator|: аналогично, но с побитовым ИЛИ и копированием «хвоста» более длинного операнда.

9. operator~: создаёт результат той же длины, инвертирует все элементы pMem. Замечание: в текущей реализации не обнуляются «лишние» биты последнего элемента, что может привести к некорректному поведению при сравнении и выводе (биты за пределами BitLen не должны учитываться). В тестах это не проявляется, так как сравнение идёт по BitLen, но при выводе через GetBit лишние биты не видны.

10. operator==: сравнивает BitLen и все элементы pMem.

11. operator=: проверяет самоприсваивание, освобождает старую память, выделяет новую и копирует.

12. operator>>: читает символы '0'/'1' до BitLen или до конца потока, устанавливает/сбрасывает биты.

13. operator<<: выводит биты от 0 до BitLen-1 в виде строки.

Класс TSet:

1. Конструкторы: инициализируют MaxPower и BitField.

   InsElem(Elem): BitField.SetBit(Elem).

   DelElem(Elem): BitField.ClrBit(Elem).

   IsMember(Elem): BitField.GetBit(Elem).

2. operator+ (с элементом): создаёт копию, добавляет элемент.

3. operator- (с элементом): создаёт копию, удаляет элемент.

4. operator+ (с множеством): возвращает TSet(BitField | s.BitField) — объединение.

5. operator*: возвращает TSet(BitField & s.BitField) — пересечение.

6. operator~: возвращает TSet(~BitField) — дополнение.

7. operator<<: выводит элементы в формате {1, 3, 5}.

4. Результаты экспериментов и тестирования

Для тестирования использовался фреймворк Google Test. Написано 61 тест, покрывающих:

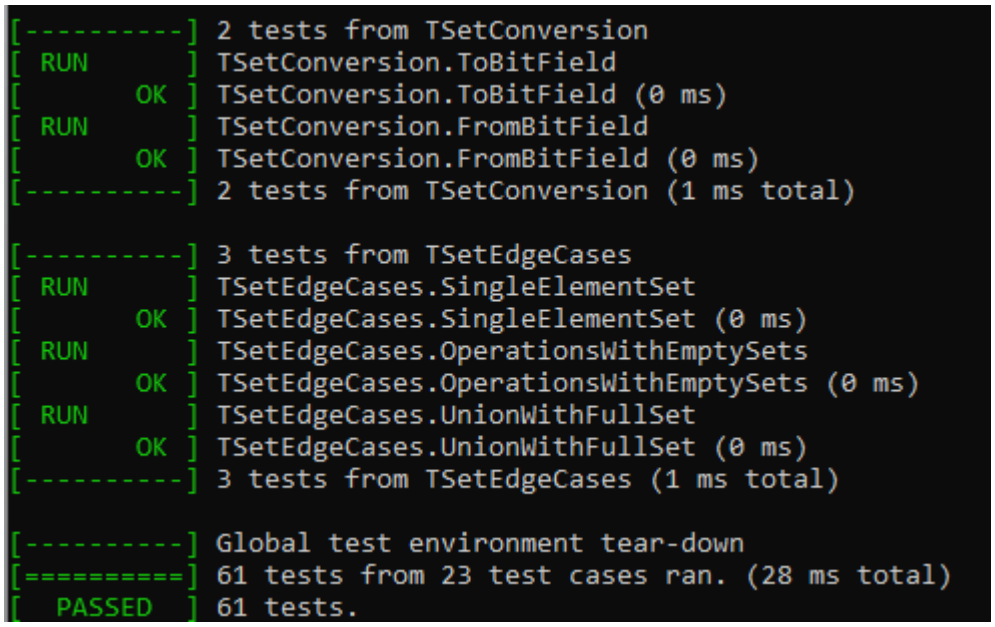
Для TBitField:

1. Конструкторы (валидные/невалидные длины, начальное состояние).
2. GetLength, SetBit, GetBit, ClrBit (включая граничные индексы и исключения).
3. Конструктор копирования и оператор присваивания (включая самоприсваивание).
4. Операторы ==, != (равные/разные поля, разные длины).
5. Операции &, |, ~ (базовые случаи, с пустым полем, двойное отрицание).
6. Ввод/вывод (формат, round-trip).
7. Краевые случаи (поле из 1 бита, граничные биты 0 и 31, последний бит последнего элемента).

Для TSet:

1. Конструкторы (валидная мощность, начальное отсутствие элементов, GetMaxPower).
2. InsElem, DelElem, IsMember (включая дублирование и все элементы).
3. Конструктор копирования и оператор присваивания.
4. Операторы ==, != (равные/разные множества, разная мощность).
5. Операторы + и - с элементом (проверка независимости исходного множества).
6. Объединение (+) и пересечение (*).
7. Дополнение (~), двойное дополнение.
8. Ввод/вывод (формат, пустое множество, round-trip).
9. Преобразование типов (TSet ↔ TBitField).
10. Краевые случаи (множество из 1 элемента, операции с пустыми множествами, объединение с полным множеством).

Пример результатов тестирования:



Все тесты проходят успешно.

Заключение

В ходе лабораторной работы были реализованы классы TBitField и TSet с полным набором конструкторов, деструкторов, операций копирования, присваивания, сравнения, побитовых и теоретико-множественных операций, а также потокового ввода/вывода. Написано более 50 модульных тестов, покрывающих все методы и граничные случаи. Все тесты проходят успешно. Получены навыки работы с динамической памятью, перегрузкой операторов, модульным тестированием и документированием кода.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация Google Test: https://google.github.io/googletest/